Last updated 28. Dezember 2021
Martin Stieben

Einführung in Python

1 Einleitung

Python ist eine universelle Programmiersprache, deren Design sich auf die Lesbarkeit und Kompaktheit von Programmen konzentriert. Da Python multiparadigmatisch arbeitet und Variablen dynamisch typisiert, wird es oft als Skriptsprache verwendet und eignet sich besonders für Programmieranfänger zum Erlernen von grundlegenden Konzepten des Programmierens.
Python Compiler und Interpreter sind für alle gängigen Betriebssysteme verfügbar:

Download: https://www.python.org/downloads
Dokumentation: https://docs.python.org/3

Diese Einführung bezieht sich auf **Python 3.9.2** und wird Ihnen einen Überblick über grundlegende Konzepte des Programmierens und deren Realisierung in Python geben. Python 2 wird seit einiger Zeit nicht mehr weiterentwickelt. Es verwendet eine etwas andere Syntax, wodurch Programme beider Versionen in der Regel nicht untereinander kompatibel sind.
Im Rahmen dieser Einführung wird Jupyter-Notebook verwendet, eine GUI für das Schreiben und Blockweise ausführen von Python-Code. Sie benötigen keine eigene Installation hiervon. Bitte verwenden Sie für diesen Kurs das PHYSnet Juypter-Hub:

```
https://jhub.physnet.uni-hamburg.de
```

Dort melden Sie sich mit Ihrer PHYSnet-Kennung an, und starten einen Jupyter-Notebook-Server mit den vorgeschlagenen Standardeinstellungen.

2 Erste Schritte mit Python

2.1 Das Jupyter-Notebook

Nachdem Sie sich erfolgreich auf dem PHYSnet Jupyter-Hub angemeldet und dort Jupyter-Notebook-Server gestartet habenn sehen Sie zu Beginn einen Verzeichnisbaum. Dieser entspricht Ihrem PHYSnet Home-Verzeichnis und ist identisch mit Ihrem Home-Verzeichnis beim Login auf einem PHYSbuntu-PC (z.B. in unseren PC-Poolräumen). Von hier aus können Sie neue Notebooks für Ihre Programme anlegen, bestehende Notebooks weiterbearbeiten oder Daten für Ihre Programme ablegen.
Sie haben außerdem die Auswahl Jupyter-Notebooks mit verschiedenen Python Versionen als "Kernelßu starten. Bitte verwenden Sie für diese Einführung den Kernel Python 3.9.3 (PHYSnet)". Ein Jupyter-Notebook ist in Blöcke von (Input-)Source-Code (*In[*]*) und der jeweils zugehörigen Rückgabe der Ergebnisse nach dem Ausführen (*Out[*]*) aufgeteilt. Zusammengehörender Input und Output ist erhält stets die selbe fortlaufende Nummerierung, wobei die Blöcke danach nummeriert werden, in welcher Reihenfolge sie ausgeführt wurden. Input-Blöcke können unabhängig voneinander mit *Shift+Return* ausgeführt werden. Wenn ein Block jedoch Daten, Variablen oder Funktionen aus einem anderen Block benötigt, muss dieser andere Block zuerst ausgeführt werden, um die benötigten Objekte verfügbar zu machen.

2.2 Grundrechenarten und Operatoren

Zahlen werden in Python entsprechend allgemeingültiger Rechenregeln verarbeitet. Wenn Sie im Jupyter-Notebook eine Rechenoperation eintippen und ausführen, wird Ihnen wie bei einem Taschenrechner das Ergebnis zurückgegeben. Weisen Sie eine Rechenoperation einer Variablen zu ($x = 2 * 3$) wird das Ergebnis nicht zurückgegeben, sondern in der Variablen gespeichert. Im Falle mehrerer Operationen in einem Block wird nur das Ergebnis der letzten Rechnung angezeigt. Hier eine Übersicht über die grundlegenden Rechenoperatoren:

Operator-Typ	Syntax	Beispiel	Ergebnis
Addition und Subtraktion	+ und -	2.0 + 3.6 - 1.2	4.4
Multiplikation und Division	* und /	3*5/2	7.5
Potenzrechnung	** oder pow(x,y)	3**2 oder pow(3,2)	9
Ganzzahldivision	//	7/2	3
Modulo (Rest nach Ganzzahldivision)	%	10%4	2

2.3 Umgang mit Variablen

Variablen sind benannte Datenfelder, welche Einträge bestimmter Datentypen enthalten können. Python3 verwaltet die Datentypen weitgehend automatisch, sodass eine Variable durch einfaches Benennen und Füllen mit einem beliebigen Wert erzeugt werden kann. Beispiele:

Syntax	Wirkung	Datentyp
x=2	erzeugt die Variable *x* und gibt ihr den Wert *2*	Integer
y=3.1	erzeugt die Variable *y* und gibt ihr den Wert *3.1*	Float
name="Maximilian"	erzeugt die Variable *name* und gibt ihr den Wert *Maximilian*	String
isFemale=False	erzeugt die Variable *isFemale* und gibt ihr den Wert *False*	Bool

Einmal definiert können Variablen weiter verwendet oder neu gefüllt werden.

```
x = 3          # x enthaelt den Wert 3
y = x+1        # x enthaelt weiterhin den Wert 3 und y=4
x = y**2       # x wird mit dem Wert 16 ueberschrieben, y bleibt 4
```

Wird ein bestimmter Dateityp benötigt, können Variablen mit *int()*, *float()* oder *str()* auch in einen andern Typ konvertiert werden. Dies ist z.B. nötig wenn man mit Werten aus einem String rechen möchte.

```
x = "3"
y = float(x)**2
```

2.4 Strings und kontrollierter Input/Output

Als sogenannte Strings werden Zeichenketten bezeichnet, welche im allgemeinen beliebige Textzeichen und Zahlen beinhalten können. Eine Zeichenkette wird mit Anführungszeichen (") als String gekennzeichnet.

```
name = "Maximilian"
```

Um eine interaktive Tastatureingabe von einem User anzufordern, kann der Befehl *input(<Text>)* verwendet werden. Der Dateityp der so gewonnenen Daten ist stets String. Der Inhalt von *<Text>* wird dem User angezeigt.

```
eingabe = input("Ihre Eingabe bitte: ")
```

Um Text interaktiv auszugeben, wird der Befehl *print()* verwendet. Hier können Textbausteine und Variablen mit Komma (,) aneinandergereiht und so eine Ausgabezeile erzeugt werden. Jedem mit Komma angehängten

Abschnitt wird im Ausgabetext eine Leerzeichen vorangestellt. Innerhalb von Stings kann der Print-Befehl auch gängige Steuerzeichen interpretieren.

Steuerzeichen	Wirkung
\n	Zeilenumbruch
\t	Tabulator

Hier ein Beispielprogramm, dass eine User-Eingabe quadriert:

```
eingabe = input("Welche Zahl soll quadriert werden?")
antwort = float(eingabe)**2
print("Das Quadrat der Zahl", eingabe, "lautet: \t", antwort)
```

2.5 Module

Python3 erlaubt das einfache Verwenden von Modulen, welche je nach Bedarf erweiterte Funktionalitäten bereitstellen. Das Einbinden von Modulen erfolgt mit dem Befehl *import* und sollte stets am Anfang des Programms geschehen. Um die Verwendung der Module im späteren Programm angenehmer zu machen, kann jedem Modul mit dem Zusatzbefehl *as* eine beliebige Alias zugewiesen werden. Oft ist es aber sinnvoller die Originalnamen der Module beizubehalten, da nicht intuitive Abkürzungen und allzu kreative Aliase schnell zu Verwirrung führen können. Wenn Sie eine bestimmte Funktionalität benötigen, recherchieren Sie zunächst, ob es eventuell bereits ein Modul dafür gibt. Hier ein kleiner Auszug häufig benötigter Module:

Name	Funktionalitäten
math	erweiterte mathematische Funktionen und Konstanten wie z.B. sin() oder pi.
random	Generieren von Zufallszahlen.
time	Datums und Zeitinformationen sowie Zeitmessungen
numpy	umfangreiches Paket für wissenschaftliche Berechnungen insb. mit Arrays.
matplotlib	Erzeugen von Grafiken aller Art.
shutil	Dateien kopieren, verschieben, etc.
os	Operationen des Betriebssystems wie z.B. Dateien löschen oder deren Existenz prüfen.

In der jeweiligen Online-Dokumentation eines Moduls finden Sie üblicherweise eine Auflistung der verfügbaren Funktionen mit kurzen Erläuterungen. Die Verwendung von Modulen erfolgt nach dem Schema *ModulName.FunktionName()*. Hier ein Beispiel, dass Kalenderdaten ausgibt und 5 Sekunden wartet:

```
# importiere time und kuerze den Modulnamen mit "t" ab
import time as t
print(t.localtime())   # Zeit ausgeben
t.sleep(5)             # 5 Sekunden warten
print(t.localtime())   # Zeit nochmal ausgeben
```

2.6 Listen

Listen sind einfache Sammelobjekte für Daten und können einer Variable zugeordnet werden. Dabei können alle Objekttypen aus Python in Listen zusammengefügt werden, solange jedes Element der Liste den selben Dateityp besitzt (z.B. nur Zahlen). Angelegt wird eine Liste mit eckigen Klammern und per Komma getrennten Elementen. Ist eine Liste erzeugt, kann diese als ganzes, als Ausschnitt (sogenanntes Slice) oder elementweise zum Auslesen oder Überschreiben verwendet werden. Die Nummerierung um Elemente zu adressieren, beginnt bei 0.

```
z = [1.2, 3, 4.6, 12, 14.9] # anlegen der Liste
print(z)        # gibt die gesamte Liste aus
print(z[0])     # gibt das 1. Element der Liste aus
z[1] = 5        # aendert den Inhalt des 2. Elements
print(z[1:3])   # gibt ein Slice vom 2. bis 4. Element aus
```

Auch Listen können Element einer Liste sein um so z.B. höherdimensionale Matrizen zu erzeugen.

```
# Anlegen einer 3x3 Matrix
z = [[1.1, 2.1, 3.1],[1.2, 2.2, 3.2],[1.3, 2.3, 3.3]]
# Ausgabe des Elements in der 1. Zeile und 2. Spalte
print(z[0][1])
```

2.7 Numpy Arrays

Eine sehr mächtige Alternative zu einfachen Listen bieten Arrays aus dem Modul *numpy*. Die Struktur von Arrays ist im Grunde identisch mit der von Listen und alle Operationen, die auf Listen anwendbar sind, funktionieren auch mit Arrays. Das Modul *numpy* bietet aber eine Vielzahl von zusätzlichen Funktionen, die den Umgang mit Arrays sehr komfortabel machen können. Ein Tutorial mit vielen nützlichen Funktionen von *numpy* finden Sie unter:

`https://docs.scipy.org/doc/numpy-dev/user/quickstart.html`

Ein wichtiges Anwendungsgebiet von *numpy* Arrays ist es z.B. eine mathematische Operation auf alle Elemente einer Liste anzuwenden. Im Falle von Listen müsste mit Hilfe einer Schleife über die Elemente iteriert und diese einzeln verändert werden. Das Multiplizieren einer Zahl x mit einer Liste würde diese sogar schlicht x-Mal hintereinander kopieren. Mit *numpy* Arrays kann hingegen wie mit Vektoren/Matrizen gerechnet werden.
Probieren Sie folgendes Beispiel, um den Unterschied zu sehen:

```
import numpy                    # numpy importieren

a = [1,2,3,4,5]                 # Anlegen einer Liste
b = numpy.array([1,2,3,4,5])    # Anlegen eines Arrays

a2 = 2*a                        # Multiplikation mit 2
b2 = 2*b

print(a2)                       # Ausgabe des Ergebnisses
print(b2)
```

2.8 Zufallszahlen

Die Generierung von Zufallszahlen ist eine der wichtigsten Operationen in den meisten physikalischen Simulationen. In Python gibt es hierfür das Modul *random*, welches viele Funktionen bietet, um unterschiedlichste Zufälligkeiten zu erzeugen. Als Grundlage für alles dient aber die Erzeugung von gleich verteilten Werten zwischen 0 und 1, wofür das Modul *random* die gleichnamige Funktion *random()* anbietet. Hier ein Beispiel:

```
import random
x = random.random()
print("Die generierte Zufallszahl lautet", x)
```

2.9 Kommentare

Kommentare sind Abschnitte in einem Programmcode, die nicht ausgeführt werden, sondern dem Verstehen und Strukturieren des Codes dienen. Es ist stets ratsam einen Programmcode so ausführlich wie möglich mit Kommentaren und Erklärungen zu versehen, damit auch für außenstehende Personen nachvollziehbar ist, was ein Programm an welcher Stelle tut. In der professionellen Softwareentwicklung besteht ein Programmcode etwa zur Hälfte aus Kommentaren. In Python3 beginnt ein normaler Kommentar stets mit einem # und gilt bis zum Ende der aktuellen Zeile. Um einen Block von Zeilen in einen Kommentar zu setzten, muss dieser mit dreifachen Anführungszeichen (“”“) eingeklammert werden.

```
y=x**2 # hier wird der Wert aus x quadriert
""" Das Quadrieren einer Zahl ist zwar einfach,
    aber trotzdem steht hier ein langer Kommentar."""
```

2.10 Übungen

1. Erstellen Sie eine 4x4 Matrix mit Listen und geben Sie die Diagonalelemente aus.
2. Schreiben Sie ein Programm, dass die Eingabe eines Anwenders von Zoll in Zentimeter umrechnet.
3. Schreiben Sie ein Programm, dass dem Anwender eine zufällige Rechenaufgabe stellt und die Antwort des Anwenders sowie die korrekte Lösung ausgibt.

3 Grundlegende Programmstrukturen

Im Gegensatz zu den meisten anderen Programmiersprachen verwendet Python keine Klammern zur Strukturierung und Kennzeichnung von Hierarchieebenen. Stattdessen beginnen neue Strukturblöcke z.B innerhalb von Verzweigungen und Schleifen mit einem Doppelpunkt (:) und umfassen alle Zeilen, die gegenüber der aktuellen Hierarchieebene um die gleiche Anzahl Leerzeichen eingerückt sind. Die erste Zeile, die nicht mehr eingerückt ist, gehört nicht mehr zu dem jeweiligen Strukturblock. Hier ist nun **besondere Vorsicht** geboten, denn auch Tabulatoren können eine Zeile eingerückt aussehen lassen, haben aber einen anderen Einfluss auf die Strukturierung. Falsche Einrückung oder Mischungen von Tabulator- und Leerzeichen sind unter Anfängern einer der häufigsten Programmierfehler in Python. Üblich ist in Python immer eine Einrückung um 4 Leerzeichen und **keine** Verwendung von Tabulator-Zeichen.

3.1 Bedingungen und logische (Boolsche) Operatoren

Um Verzweigungen und Schleifen steuern zu können müssen entsprechende Bedingungen formuliert werden. Eine Bedingung ergibt immer einen boolschen Wert (*True* oder *False*) kann über logische Operatoren beliebig mit weiteren Bedingungen zu einem Gesamtergebnis *True* oder *False* verknüpft werden. Typische Bedingungen sind folgende Vergleiche:

Operator-Typ	Syntax	Beispiel	Ergebnis
größer und kleiner als	> und <	5 < 8	true
größer und kleiner gleich	>= und =<	5 <= 3	false
exakt gleich	==	“Text1“ == “Text2“	false
ungleich	!=	“Text1“ != “Text2“	true

Weitere mögliche Bedingungen sind nicht mehr allgemeingültig und werden im später in dem Kontext erwähnt, in dem sie Anwendung finden können.
Zur Verknüpfung von boolschen Werten stehen Ihnen drei logische Operatoren zur Verfügung:

Operator	Wirkung
and	beide Bedingungen müssen erfüllt sein, um *True* zu ergeben
or	mindestens eine Bedingung muss erfüllt sein, um *True* zu ergeben
not	invertiert den boolschen Wert

Dabei ist es egal, ob die verknüpften Werte aus Bedingungen kommen, Variablen mit boolschem Wert oder explizite Werte sind. Um mehrere Verknüpfungen durchzuführen, können diese einfach aneinandergereiht werden. Mit einfachen Klammern () umschlossene Verknüpfungsketten werden dabei zuerst ausgewertet.

3.2 Verzweigungen

Eine Verzweigung mit dem Operator *if* erlaubt das bedingungsgesteuerte Ausführen eines bestimmten Code-Blocks. Optional kann mit dem Folgeopterator *elif* eine alternative Bedingung geprüft werden oder mit *else* eine bedingungslose Alternative angeboten werden. Wird keine Alternative implementiert, und das Ergebnis der If-Bedingung lautet *False*, wird der entsprechende Code-Block übersprungen. Hier ein Beispielprogramm, welches das Vorzeichen einer Zahl auswertet:

```
x = float(input("Geben sie eine beliebige Zahl ein"))
if x > 0:
    print("Diese Zahl ist positiv")
elif x < 0:
    print("Diese Zahl ist negativ")
else:
    print("Diese Zahl ist gleich 0")
```

3.3 Schleifen

Schleifen ermöglichen das wiederholte Ausführen eines bestimmten Code-Blocks. Zur Steuerung der Wiederholungsanzahl einer Schleife gibt es zwei unterschiedliche Konzepte:

1. *for*-Schleife:
   Eine *for*-Schleife iteriert über eine regelmäßige Folge von Werten, deren Inhalte und Struktur bereits vor dem Beginn der Schleife bekannt sind. Sie wird daher als Zählschleife bezeichnet.

2. *while*-Schleife:
   Eine *while*-Schleife wird verwendet, wenn sich jeweils erst durch die Ausführung des entsprechenden Code-Blocks ergibt, ob die Schleife ein weiteres Mal ausgeführt werden muss. Diese sogenannte bedingungsgesteuerte Schleife arbeitet also mit einer vorher unbekannten Anzahl an Durchläufen auf die Erfüllung eines Abbruchkriteriums hin.

Beide Schleifen können mit Vergleichen als Bedingungen gesteuert werden. Die *for*-Schleife bietet aber auch die Möglichkeit über eine Folge von beliebigen Elementen (z.B. die Einträge in einer Liste) zu iterieren. Hierzu dient die Befehlskombination *for <Iterator> in <List>*. Der Interator *i* ist dabei während der Schleife als Variable verfügbar und enthält bei jedem Durchlauf den nächsten Wert aus der Liste. Ein Beispiel:

```
liste = [2,-3,10,-5,22]
for i in liste:
    print ("aktuelle Zahl:", i)
```

Für die *while*-Schleife wird typischerweise als Steuerbedingung ein Vergleich verwendet. Überprüft wird ein Wert, der durch die Schleife selbst verändert wird, um die Schleife irgendwann beenden zu können.

```
schritte = 0
while schritte < 10:
    schritte = schritte + 1
    print ("Summe der bisherigen Schritte:", schritte)
```

Schleifen können zudem an jeder Stelle mit einem *break*-Befehl vollständig abgebrochen werden, während mit einem *continue*-Befehl lediglich der aktuelle Durchgang der Schleife übersprungen wird. Damit diese Kontrollbefehle ihren Zweck erfüllen können, müssen sie z.B über eine *if*-Anweisung an eine Bedingung geknüpft werden. Das folgende Programmbeispiel bricht ab (*break*), sobald die Zahl 10 auftritt:

```
liste = [2,-3,10,-5,22]
for i in liste:
    if i == 10:
        break
    print ("aktuelle Zahl:", i)
print("Ende.")
```

In Gegensatz dazu gibt das nächste Programmbeispiel nur die positiven Zahlen der Sequenz aus und überspringt (*continue*) alle anderen:

```
liste = [2,-3,10,-5,22]
for i in liste:
    if i < 0:
        continue
    print ("aktuelle Zahl:", i)
print("Ende.")
```

3.4 Funktionen

Funktionen dienen dazu ein Programm zu modularisieren, also Code-Blöcke, die eine bestimmte Aufgabe erfüllen als Einheit einzukapseln. Bei größeren Programmen erhöht dies die Übersichtlichkeit erheblich aber der größte Vorteil liegt darin, mehrfach benötigte Befehlsketten nur einmal aufschreiben zu müssen. So ist es auch möglich, Funktionen von anderen Programmierern zu verwenden. Die bereits erwähnten Module für Python3 tun nichts anderes, als Ihnen vorgefertigte Funktionen zur Verfügung zu stellen, die andere Programmierer bereits implementiert und optimiert haben.
Um eine Funktion Verwenden zu können, muss diese zu Beginn des Programms einmalig definiert werden.

```
#definiere die Funktion
def trennlinie():
    print("--------------------")

#Hauptprogramm
liste = [2,-3,10,-5,22]
for i in liste:
    print ("aktuelle Zahl:", i)
    trennlinie()
```

Natürlich können Funktionen auch innerhalb von anderen Funktionen aufgerufen werden, die im Programm weiter unten definiert werden. Das Hauptprogramm muss in Python im Gegensatz zu vielen anderen Programmiersprachen **nicht** in einer Funktion *main()* eingebettet werden.
Einer Funktion können auch sogenannte Parameter mitgegeben werden, welche in der Funktion verarbeitet werden sollen. Diese Parameter werden bei der Definition der Funktion benannt und stehen unter diesem Namen innerhalb der Funktion als Variable zur Verfügung.

```
#definiere die Funktion
def squareAndPrintThisValue(aktuellerWert):
    quadrat = aktuellerWert**2
    print ("Zahl:", aktuellerWert, " Quadrat:", quadrat)

#Hauptprogramm
liste = [2,-3,10,-5,22]
for i in liste:
    squareAndPrintThisValue(i)
```

Die bisherigen Beispiele für Funktionen führen eine Aufgabe aus, ohne sich beim Hauptprogramm zurückzumelden. Eine solche Funktion hat den Typ *void*. Oft werden Funktionen aber auch dazu benötigt, ein bestimmtes Ergebnis zu generieren, welches in z.B. Hauptprogramm weiter verwendet werden soll. Diese Rückgabe eines Ergebnisses erfolg mit dem Befehl *return*. Das folgende Programmbeispiel nutzt eine Funktion zum quadrieren einer Zahl:

```
#definiere die Funktion
def squareThisValue(aktuellerWert):
    ergebnis = aktuellerWert**2
    return ergebnis

#Hauptprogramm
liste = [2,-3,10,-5,22]
for i in liste:
    quadrat = squareThisValue(i)
    print ("Zahl:", i, " Quadrat:", quadrat)
```

3.5 Fehler abfangen

Das Abfangen von Fehlern ist eine der wichtigsten Aufgaben beim Entwickeln von Programmen, die interaktiv insbesondere mit Personen arbeiten sollen. So wird sichergestellt, dass eine interaktive Eingabe auch tatsächlich weiterverarbeitet werden kann, ohne das Programm abstürzen zu lassen. Hierzu dienen die Befehle *try* und *except*. Ähnlich wie bei *if* und *else* wird zunächst versucht den Code-Block innerhalb von *try* auszuführen. Sollte hier ein Fehler auftreten, wird stattdessen der Code-Block innerhalb von *except* ausgeführt. Im folgenden Programmbeispiel wird der User solange nach einer neuen Eingabe gefragt, bis er eine ganze Zahl eingegeben hat:

```
# Die Variable "fehler" dient dazu, die Schleife aufrecht
# zu erhalten, bis die Eingabe erfolgreich war.
fehler = 1

# Schleife zur Wiederholung der Eingabe
while fehler == 1:
   eingabe = input("Bitte geben sie eine ganze Zahl ein.")

       # Versuche die Eingabe als ganze Zahl (Int) zu verwenden.
       try:
          zahl = int(eingabe)
          print("Super!", zahl, "ist eine ganze Zahl.")
          # zum Verlassen der Schleife
          fehler = 0
       except:
          print(eingabe, "ist keine ganze Zahl. Nochmal!")
```

3.6 Übungen

1. Schreiben Sie ein Programm, dass dem Anwender eine zufällige Rechenaufgabe stellt und die Antwort des Anwenders auf Korrektheit überprüft. Der Anwender soll solange wiederholt nach einer Antwort gefragt werden, bis seine Antwort korrekt ist.

2. Schreiben Sie ein Programm, dass den Mittelwert von Einträgen in einer Liste berechnet. Erstellen Sie hierfür eine Funktion, die als Parameter eine Liste erhält und den Mittelwert zurückgibt. Vergleichen Sie das Ergebnis Ihrer Funktion mit dem Ergebnis von numpy.average(<liste>).

4 Externe Daten

Python bietet eine Vielzahl von Interaktionsmöglichkeiten mit externen Daten. Elegant ist dabei die Verwendung von Datenbanken in MySQL oder SQLite, doch in der Physik wird sehr oft mit Daten in strukturierten Textdateien gearbeitet. Diese Struktur wird CSV (Comma-separated Values) genannt und bildet eine Tabelle. Hierbei entsprechen die Zeilen der Tabelle den Zeilen der Textdatei, währen die Spalten durch ein einheitliches Trennzeichen festgelegt werden. Als Trennzeichen wird üblicherweise ein Komma (daher auch der Name CSV), Semicolon, oder Tabulator verwendet, wobei theoretisch jedes Zeichen als Trennzeichen verwendet werden könnte. Wichtig ist dabei nur, dass dieses Zeichen in den eigentlichen Daten nicht vorkommt.
Im Rahmen dieser Einführung werden wir uns auf solche CSV-Dateien konzentrieren.

4.1 Öffnen und Schließen von Dateien

Die in Python integrierte Funktion *open()* erlaubt das Bereitstellen einer externen Datei als Dateiobjekt im Programm. Dazu benötigt *open()* den Pfad und Namen der gewünschten Datei als Argument. Sowohl absolute, also auch relative Pfade sind möglich, wobei ein relativer Pfad stets vom Ort Ihres Python-Programms ausgeht. Der Rückgabewert von *open()* wird einer Variable übergeben.

```
# oeffnet eine Datei "test.csv" aus dem Unterordner "daten".
inputData = open("daten/test.csv")
```

Neben dem Pfad kann der *open()*-Funktion auch ein Öffnungsmodus als zweiter Parameter mitgegeben werden. Dieser Modus legt fest, welche Operationen mit der Datei im Programm erlaubt sind.

```
# oeffnet eine Datei "test.csv" im read-only modus
inputData = open("test.csv","r")
```

Hier ist eine Auflistung der wichtigsten Modi:

Modus	Wirkung
r	nur zum Lesen öffnen (Standard, wenn kein Modus angegeben wird)
w	nur zum Schreiben öffnen (Datei wird neu erzeugt oder überschrieben)
a	nur zum Schreiben öffnen (Am Ende der Datei weiterschreiben)
r+	zum Lesen und Schreiben öffnen (Position am Anfang der Datei)
w+	zum Lesen und Schreiben öffnen (Datei wird beim Öffnen geleert)
a+	zum Lesen und Schreiben öffnen (Position am Ende der Datei)

Wenn eine Datei bearbeitet wurde, muss diese am Ende des Vorgangs mit dem Befehl *close()* wieder geschlossen werden. Andernfalls werden die Änderungen nicht übernommen oder die Datei wird beschädigt und wäre nicht mehr lesbar.

```
inputData = open("test.csv") # Datei oeffnen
# etwas mir der Datei tun ...
inputData.close() # Datei schliessen
```

4.2 Sequentielles Lesen und Schreiben

Wurde eine Datei in einem Schreibmodus geöffnet, so kann mit der Funktion *write()* Inhalt hinzugefügt werden. Dabei werden auch hier Steuerzeichen (siehe Kapitel 2.4) interpretiert. *write()* muss als Argument Strings erhalten, sodass andere Dateitypen zunächst konvertiert werden müssen. Mehrere Textabschnitte können entweder mit "+" verknüpft oder mit dem Befehl *writelines()* als Liste übergeben werden. Bedenken Sie, dass *write()* **keinen** automatischen Zeilenumbruch erzeugt. Hier ein Beispiel:

```
data = open("data.csv","w") # Datei im Schreibmodus oeffnen

# Eine Zeile schreiben (\n = Zeilenumbruch)
data.write("Dies ist die erste Zeile.\n")

# Eine Folge von Textbloecken schreiben
x = 2
data.write("Dies ist Zeile Nr. " + str(x) + ".\n")

# Eine Liste von Textbloecken schreiben
z = ["Zeile 3...\n", "Zeile "+str(x+2)+"...\n", "weitere Zeilen"]
data.writelines(z)

data.close() # Datei schliessen
```

Zum Einlesen einer Datei gibt es mehrere Funktionen, die unterschiedliche Strategien erlauben, je nachdem, welche Struktur die Daten haben und wie diese verwendet werden sollen.
Der Befehl *readline()* liest nur eine einzige Zeile als String ein. Dabei wird im Hintergrund einem Iterator gefolgt, sodass das erneute Aufrufen von readline() die jeweils nächste Zeile einliest. Ist das Ende der Datei erreicht, würden nur noch leere Strings eingelesen werden.

```
data = open("data.csv","r") # Datei im Lesemodus oeffnen

print(data.readline()) # liest die erste Zeile und gibt sie aus
print(data.readline()) # liest die zweite Zeile und gibt sie aus
# usw.

data.close() # Datei schliessen
```

Der Befehl *readlines()* (Achtung: Plural!) liest gleichzeitig alle Zeilen einer Datei als Liste ein. Jedes Element der Liste enthält eine Zeile als String. Folgenses Beispiel liest eine Datei als Liste ein und gibt alle Elemente der Liste nacheinander aus:

```
data = open("data.csv","r") # Datei im Lesemodus oeffnen
zeilenliste = data.readlines() # als Liste einlesen

# iteration ueber die Liste mit "zeile" als Iterationsvariable
for zeile in zeilenliste:
    print(zeile)

data.close() # Datei schliessen
```

Der Befehl *read()* liest die gesamte Textdatei als einen einzigen String ein. Somit wird es nötig, diesen String nach dem Einlesen mit dem Befehl *split()* in eine Liste zu zerlegen. Das gleiche gilt auch für Zeilen einer CSV-Datei, die mehrere Spalten haben, die z.B. per Semikolon getrennt sind. *split()* erhält als Argument ein Trennteichen. Bei jedem Auftreten des Trennzeichens, wird der String zerschnitten und das Trennzeichen entfernt. Wird kein explizites Trennzeichen angeben, trennt *split()* automatisch nach Leerzeichen und Zeilenumbrüchen.

```
data = open("data.csv","r") # Datei im Lesemodus oeffnen
ganzeDatei = data.read() # ganze Datei als einzigen String einlesen

# automatisch trennen (jedes Wort) und Liste ausgeben
wortliste = ganzeDatei.split()
for wort in wortliste:
    print(wort)

# nur bei Zeilenumbruch "chr(10)" trennen
zeilenliste = ganzeDatei.split(chr(10))
for zeile in zeilenliste:
    print(zeile)

data.close() # Datei schliessen
```

4.3 Daten einlesen mit Numpy

Auch das Modul *numpy* bietet die Möglichkeit, eine CSV-Datei direkt in ein Array einzulesen. Dafür müssen die Daten innerhalb der Datei aber bestimmte Kriterien erfüllen. Einerseits müssen die Spalten in einer Zeile mit Tabulatoren getrennt sein und andererseits dürfen keine leeren Zeilen oder unterschiedliche Datentypen (z.B. Text in einer Datei mit sonst nur Zahlen) auftreten.
Im Gegenzug müssen die eingelesenen Daten nicht mehr manuell gesplittet werden und Zahlen müssen nicht mehr manuell von einem String in einen Integer bzw. Float-Typ konvertiert werden. Hier eine Beispiel-Funktion, die eine Datei über Numpy einliest und als Array zurückgibt.

```
def datenEinlesen(dateiname):
    print("Lese Daten aus der Datei", dateiname)
    datei = open(dateiname)
    data = np.loadtxt(datei)
    return data
```

4.4 Übungen

1. Schreiben Sie ein Programm, dass 10 Paare von Zufallszahlen (x,y) erzeugt und diese in eine Datei schreibt. Jedes Paar soll in eine eigene Zeile geschrieben und mit einem Semikolon getrennt werden.

2. Schreiben Sie ein neues Programm, das die Zahlenpaare aus der Datei aus der vorherigen Übung einliest, addiert und in folgendem Format ausgibt: "Die Summe von x und y lautet z."

5 Grafische Darstellung von Daten

Zum Erzeugen von Plots dient in Python das Modul matplotlib.pyplot, welches vor der Verwendung importiert werden muss. Wird nun ein Plot erzeugt und angezeigt, so wird dieser standardmäßig in einem neuen Fenster dargestellt und das Programm an dieser Stelle angehalten, bis das Fenster wieder geschlossen wurde. Insbesondere in einem Jupyter-Notebook ist es aber wesentlich komfortabler, den Plot inline nach dem entsprechenden Input-Block anzeigen zu lassen. Hierfür dient der Konfigurationsbefehl *"%matplotlib inline"*.

```
# importiere matplotlib und kuerze den Modulnamen mit "plt" ab
import matplotlib.pyplot as plt
%matplotlib inline
```

Eine ausführliche Dokumentation zu diesem Modul mit vielen Beispielen und Anwendungsmöglichkeiten finden Sie unter:

`http://matplotlib.org/api/pyplot_api.html`

5.1 Einfache 2D-Plots

Zum Erzeugen von 2D-Plots dient der Befehl *plot()*. Dieser erwartet als Argumente je eine Liste der x- und y-Werte, sowie mehrere optionale Style-Kommandos. Um einen Plot tatsächlich anzeigen zu lassen wird der Befehl *show()* verwendet.

```
import matplotlib.pyplot as plt

x = [1,2,3,4,5]
y = [1,4,9,16,25]

plt.plot(x, y) #zeichnet eine Linie
plt.show()
```

Anstatt sich die Bilder online anzeigen zu lassen, können diese mit dem Befehl *savefig()* in einem gängigen Dateiformat (png, jpg, tiff, pdf, eps) abgespeichert werden. *savefig()* erwartet als Argument einen Dateinamen inklusive Dateiendung als String.

```
plt.plot(x, y) #zeichnet eine Linie
plt.savefig("plot.png")
```

Die Achsenskalierung und deren Beschriftungen werden mit den Befehlen *axis()* bzw. *xlabel()* und *ylabel()* formatiert. axis() erwartet als Argument eine Liste mit den Anfangs- und Endwerten für x- und y-Achse, während *xlabel()* und *ylabel()* einen String benötigen. Hier ein Beispiel:

```
plt.plot(x, y)
plt.axis([0,6,0,30])
plt.xlabel("Dies ist die x-Achse")
plt.ylabel("Dies ist die y-Achse")
```

Die Funktionen *grid()* und *text()* erlauben das Hinzufügen eines Hilfsgitters bzw. eines Textfeldes an beliebiger Stelle im Bild. *grid()* erwartet als Argument ein Boolsches *True* oder *False* zum An-/Ausschalten des Hilfsgitters. *text()* erwartet als Argumente die Koordinaten der linken unteren Ecke des Textfeldes, sowie den Text als String.

```
plt.plot(x, y)
plt.grid(True)
plt.text(1,25,"Hier steht ein Text")
```

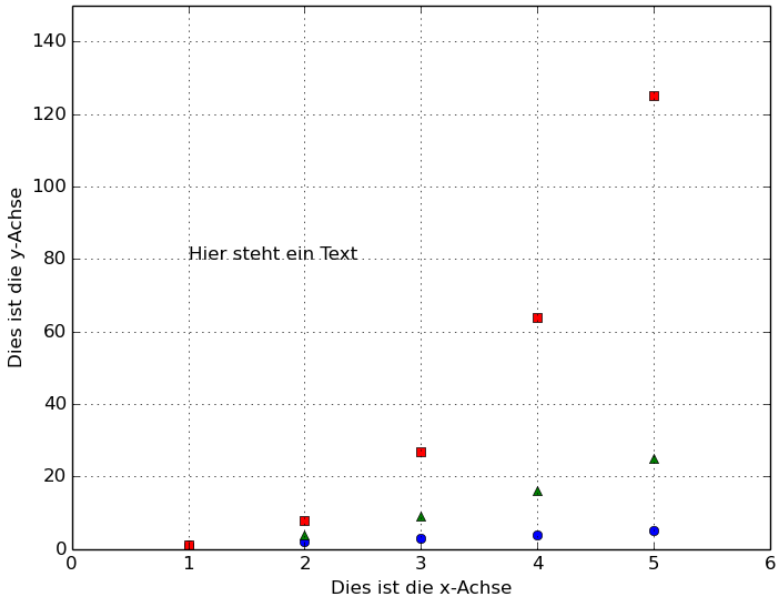
Um mehrere Kurven in einen Plot zu zeichnen, können die entsprechenden Werte und Optionen aneinandergereiht werden. Es gibt sehr viele Optionen um die Linien und ggf. Marker für die Punkte zu formatieren. Für Details schauen Sie bitte in die Modul-Dokumentation.
Hier ein vollständiges Beispiel mit dem zugehörigen Output:

```
# importiere matplotlib und kuerze den Modulnamen mit "plt" ab
import matplotlib.pyplot as plt

# erzeuge die Datenlisten fuer drei Kurven
x = [1,2,3,4,5]
x2 = [1,4,9,16,25]
x3 = [1,8,27,64,125]

plt.plot(x, x, "bo", x, x2, "g^", x, x3, "rs") # plotten
plt.axis([0,6,0,150])                          # Achsen skalieren
plt.xlabel("Dies ist die x-Achse")             # Achsen benennen
plt.ylabel("Dies ist die y-Achse")
plt.grid(True)                                 # Grid aktivieren
plt.text(1,80,"Hier steht ein Text")           # Textfeld

# Plot anzeigen
plt.show()
```



5.2 Übung

Schreiben Sie ein Programm, dass die Funktionswerte der trigonometrischen Funktionen Sinus und Cosinus mit einigen Stützstellen im Wertebereich von 0 bis 10 berechnet. Diese sollen als Linien *und* als Marker in eine gemeinsame Grafik geplottet und als PNG-Bild abspeichert werden.

5.3 Histogramme

Histogramme dienen der graphischen Darstellung von Häufigkeitsverteilungen. Dazu werden Daten in sogenannten Bins gruppiert und die Einträge in diesen Bins hochgezählt.
Die Funktion *hist()* erzeugt solche Histogramme aus einer Liste mit Datenwerten. Diese Liste enthält **nicht** die Höhe der Bins, sondern die tatsächlichen Daten, welche von der Funktion automatisch in die Bins einsortiert werden. Als Argumente erwartet hist() mindestens eine Datenliste und die Anzahl der Bins auf dem gegebenen Datenbereich, in die die Daten sortiert werden sollen.

```
# dlist ist eine Liste mit Daten, die in 50 Bins sortiert werden
plt.hist(dlist, 50)
```

Alternativ können Sie auch eine Liste der Bins und deren Höhe angeben.

```
# xlist ist eine Liste mit Bins und ylist eine mit deren Hoehen
plt.hist(xlist, weights=ylist)
```

Weitere Argumente zum Formatieren der Grafik können Sie der Modul-Dokumentation entnehmen.

In dem folgenden Beispiel mit zugehörigem Output wird eine Liste mit einer Million Gaußverteilten Zufallszahlen erzeugt und als Histogramm geplottet. Als Liste wird hier wegen der erweiterten Funktionspalette ein Array aus dem Modul *numpy* verwendet. Mit einer normalen Liste z.B. aus einer eingelesenen Datei funktioniert der Plot aber identisch.

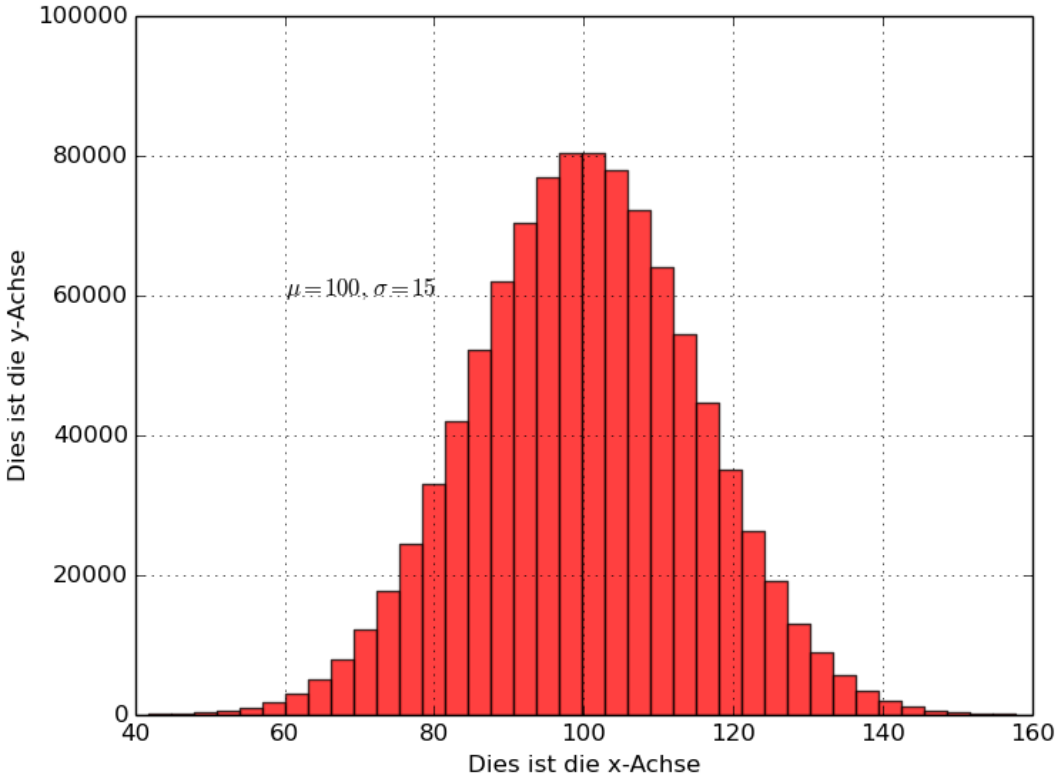
```
# importiere numpy und matplotlib
import numpy
import matplotlib.pyplot as plt

# erzeuge gaussverteilte Zufallszahlen in einem Array
mu = 100
sigma = 15
x = mu + sigma * numpy.random.randn(1000000)

# plotte ein Histogramm mit 50 Bins, roten Balken
# und 25%iger Transparenz.
plt.hist(x, 50, facecolor='r', alpha=0.75)

# Formatiere den Plot
plt.axis([40,160,0,100000])
plt.xlabel("Dies ist die x-Achse")
plt.ylabel("Dies ist die y-Achse")
plt.grid(True)
plt.text(60, 60000, r'$\mu=100,\ \sigma=15$')

# Plot anzeigen
plt.show()
```


100000
80000
60000
40000
20000
0
40
60
80
100
120
140
160
Dies ist die y-Achse
Dies ist die x-Achse
$\mu = 100, \sigma = 15$

6 Abschlussübung: π-Approximation mit Monte-Carlo-Algorithmus

Zum Abschluss dieses Grundkurses schreiben Sie ein Simulationsprogramm, dass die Zahl π schrittweise annähert. Hierzu würfeln Sie Paare von Zufallszahlen (x,y) zwischen 0 und 1, also in einem Quadrat mit der Kantenlänge 1, und berechnen den Abstand zum Koordinatenursprung $r = \sqrt{x^2 + y^2}$. Nun zählen Sie, wie viele Koordinaten sie gewürfelt haben und wie viele davon auf der Viertelkreis-Fläche $r <= 1$ liegen. Der Quotient aus beiden Zahlen und einem Faktor 4 entspricht dann der Zahl π.

$$\frac{4 * TrefferimKreis}{TrefferimQuadrat} = \frac{4 * Viertelkreisflaeche}{Quadratflaeche} = \frac{\pi * r^2}{r^2} = \pi$$

Je größer die Statistik, umso ähnlicher wird der so approximierte Wert dem tatsächlichen Wert von
$\pi = 3.14159265358979323846264338327950288419716939937 5...$

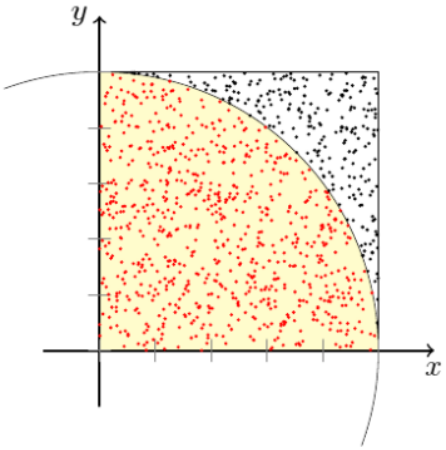


Abbildung 1: Monte-Carlo Approximation von π

1. Schreiben Sie ein Programm, dass π nach der oben beschriebenen Methode annähert. Das Würfeln der Zufallszahlen und der Test $r <= 1$ sollen in einer eigenständigen Funktion geschehen, die als Ergebnis 1 = Treffer oder 0 = kein Treffer zurückgibt. Der Benutzer soll gefragt werden, wie viele Tests durchgeführt werden sollen. Nach Durchlauf der Simulation soll dem Benutzer der für π ermittelte Wert und der Literaturwert ausgegeben werden.

2. Verändern Sie das Programm aus der vorherigen Übung so, dass die Simulation solange läuft, bis Pi auf 3 Nachkommastellen genau mit dem Literaturwert übereinstimmt.

3. **Bonus:**
   Lassen Sie Pi mehrfach auf 3 Nachkommastellen genau bestimmen und speichern Sie z.B. in einer Liste oder einer Datei ab, wie viele Zufallszahlenpaare dafür erzeugt werden mussten. Berechnen Sie von diesen Werten den Mittelwert und die Standardabweichung. Wie verändert sich die relative Standardabweichung mit zunehmender Statistik?

7 Lösungen

Sie können diese Einführung als PDF, sowie die Lösungen zu allen Übungen in dieser Einführung als Jupyter-Notebook hier herunterladen:
`https://wolke.physnet.uni-hamburg.de/index.php/s/gEL32iDDrneRWyk`